OpenROAD – Build, Release & Onboarding Simplification —

Expected Size: 175 hours
Mentor: **Jack , Vitor**

Soumyadip Das Mahapatra

soumyagle@gmail.com

github.com/Divinesoumyadip

JIS University, B.Tech CSE (4th Year)

Kolkata, West Bengal, India

My Pre-Gsoc Contributions :

PR		Repository	What I Fixed	Status
#9867	→	OpenROAD	macOS dependency installer fixes	Merged
#9821	→	OpenROAD	Build infrastructure improvement	Merged
#9819	→	OpenROAD	Dependency script fix	Merged
#9805	→	OpenROAD	Build.sh missing dependency error fix	Merged
#4030	→	OpenROAD-flow-scripts	hw.ncpu → hw.logicalcpu macOS bug fix	Merged
#4031	→	OpenROAD-flow-scripts	Windows Docker documentation	Under Review

Pre Task Submission :

I have read and comply with the OpenROAD Initiative GSoC 2026 Engagement and AI Policy.

ORFS installation completed, nangate45/gcd design run successfully on 20 March 2026, and screenshots submitted via the OpenROAD Contributors Agreement form.

Abstract

OpenROAD is one of the most important open-source **EDA tools** available today. But getting it installed and running is harder than it should be. Pre-built binaries only exist for **Ubuntu 20.04 and 22.04**. **macOS** users must build from source and a **2-hour process that frequently fails. Rocky Linux** is **unsupported**. There is no automated **release pipeline**, no semantic versioning, and **no beginner tutorial** showing **a new user** how to run their **first design**.

I have been **contributing to OpenROAD for the past several months** and hit every single one of these **pain points personally.** I didn't just read about them , **I fixed them.**

This project will complete **what I started** , a formal release **pipeline** with automated binary generation for all supported platforms, semantic versioning, and an end-to-end tutorial guiding users **from installation to their first GDSII layout.**

Every year **hundreds of students and researchers try to set up OpenROAD** for the first time and fail. They hit missing dependencies, **broken macOS builds, no Windows guide**, **no tutorial showing them what to do next**.

Most of them give up before **running** a **single design**. **This project fixes that. A first-time contributor should be able to go from zero to running their first GDSII layout in under 10 minutes** regardless of whether they are on **Ubuntu, macOS, Windows, or Rocky Linux.**

Problem Statement

Based on direct **hands-on study of both** the **OpenROAD and OpenROAD-flow-scripts** repositories, **the following concrete gaps exist today:**

1. No Automated Release Workflow:

Neither **OpenROAD nor OpenROAD-flow-scripts** has a release workflow. Releases are created manually by **Precision Innovations** with no automated changelog, no versioned tagging, no reproducible pipeline. Users cannot pin to a stable version.

```
black.yaml                                          github-actions-manual-update-rules.yml
github-actions-cron-sync-fork-from-upstream.yml     github-actions-on-delete-cleanup.yml
github-actions-cron-test-installer.yml              github-actions-on-label-create.yml
github-actions-cron-update-OR.yml                   github-actions-on-push.yml
github-actions-cron-update-yosys.yml                github-actions-publish-docker-images.yml
github-actions-cron-util-test.yml                   github-actions-update-rules.yml
github-actions-lint-tcl.yml                         github-actions-yaml-test.yml
```

2. Pre-built Binaries Cover Only 2 Platforms:

The OS matrix in `github-actions-publish-docker-images.yml` shows only two platforms are **supported:**

```yaml
    fail-fast: false
    matrix:
      os: [["ubuntu20.04", "ubuntu:20.04"], ["ubuntu22.04", "ubuntu:22.04"]]
  runs-on: ${{ vars.USE_SELF_HOSTED == 'true' && 'self-hosted' || 'ubuntu-latest' }}
  steps:
```

Rocky Linux 8/9, **macOS, and Windows** are completely absent despite being widely used in academic and industry environments.

3. macOS Build Friction:

`DependencyInstaller.sh` **has macOS support** `via` `_install_darwin_packages()` **.(I identified and fixed already)**

```bash
if [[ "$OSTYPE" == "darwin"* ]]; then
    numThreads=$(sysctl -n hw.ncpu)
else
    numThreads=$(nproc)
fi

_versionCompare() {
    local a b IFS=. ; set -f
    printf -v a %08d $1; printf -v b %08d $3
    test $a "$2" $b
}
```

Additionally `github-actions-macos.yml` only runs on forks and macOS breakage goes undetected on the main repository.

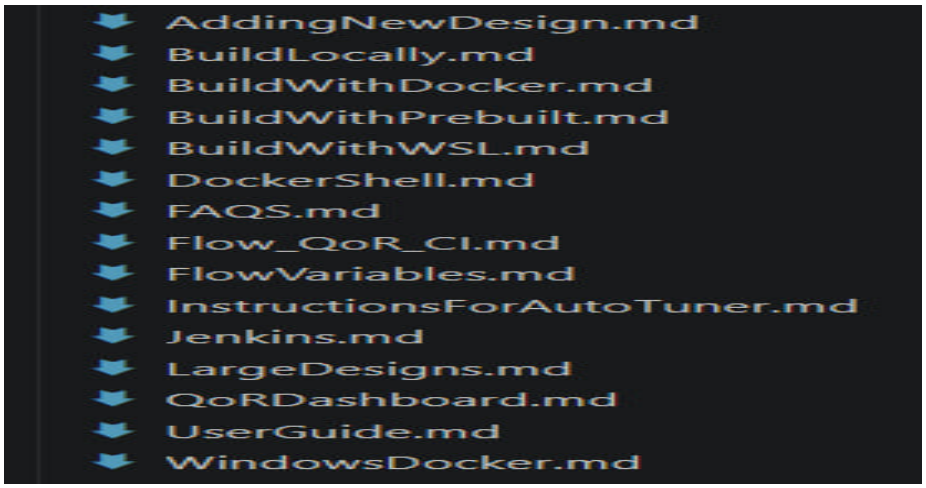
4. No Formal Versioning:

```
code /mnt/d/OpenROAD-flow-scripts/
```

No VERSION file, no semantic versioning, no `RELEASE.md`. Users cannot pin to a **stable version**.

5. Missing Platform Documentation:

`BuildWithPrebuilt.md` documents only **Ubuntu and Debian**. **Rocky Linux, macOS, and Windows have no installation guide** , despite **`DependencyInstaller.sh`** having partial **macOS support via `_install_darwin_packages()`** .



6. No Beginner Tutorial:

No guided path from installation to running first design exists in either repository.

Analysis of Current State

I studied both repositories in depth before writing this proposal. Here is exactly what exists today and what is missing.

```
63:        os: [["ubuntu20.04", "ubuntu:20.04"], ["ubuntu22.04", "ubuntu:22.04"]]
107:        os: ["ubuntu20.04", "ubuntu22.04"]
AddingNewDesign.md  BuildWithPrebuilt.md  FAQS.md           InstructionsForAutoTuner.md  QoRDashboard.md
BuildLocally.md     BuildWithWSL.md       FlowVariables.md  Jenkins.md                   UserGuide.md
BuildWithDocker.md  DockerShell.md        Flow_QoR_CI.md    LargeDesigns.md              WindowsDocker.md
15:    numThreads=$(sysctl -n hw.ncpu)
```

Technical Implementation:

The existing **`github-actions-publish-docker-images.yml`** is the foundation. It already has three jobs , buildCodespaceImage, buildDependenciesImage, buildORFSImage ,but only supports **Ubuntu**. I will extend this infrastructure rather than build from scratch.

1) Automated Release Workflow:

I will create **`github-actions-release.yml`** extending the existing buildDependenciesImage job. The trigger changes from file-path push to version tag push. The OS matrix expands from Ubuntu-only to include **Rocky Linux 8 and 9**

```
.github
  ISSUE_TEMPLATE
  workflows
    black.yaml
    github-actions-cron-sync-fork-from-upstr...
    github-actions-cron-test-installer.yml
    github-actions-cron-update-OR.yml
    github-actions-cron-update-yosys.yml
    github-actions-cron-util-test.yml
    github-actions-lint-tcl.yml
    github-actions-manual-update-rules.yml
    github-actions-on-delete-cleanup.yml
    github-actions-on-label-create.yml   4
    github-actions-on-push.yml
    github-actions-publish-docker-ima...   9+
    github-actions-release.yml   2
    github-actions-update-rules.yml
```

```yaml
on:
  push:
    tags:
      - 'v*.*.*'

jobs:
  buildDependenciesImage:
    matrix:
      os: [
          ["ubuntu20.04", "ubuntu:20.04"],
          ["ubuntu22.04", "ubuntu:22.04"],
          ["rockylinux8", "rockylinux:8"],   # NEW
          ["rockylinux9", "rockylinux:9"],   # NEW
      ]
    runs-on: ubuntu-latest
```

This reuses the existing buildDependenciesImage job structure extending the **OS matrix from 2 to 4 platforms.**

2) Rocky Linux Support:

I will add **docker/Dockerfile**.rockylinux following the same pattern as existing docker/Dockerfile.dev for Ubuntu. **Rocky Linux** uses dnf instead of apt , the only structural difference. I will also extend **`DependencyInstaller.sh with _installRockyLinux()`** function following the **`existing _install_ubuntu_packages() pattern`** .

```
bazel
  install.sh
  README.md
docker
  Dockerfile.builder
  Dockerfile.dev
  Dockerfile.rockylinux
```

```dockerfile
ARG fromImage=rockylinux:8
FROM $fromImage

RUN dnf install -y epel-release && \
    dnf install -y gcc gcc-c++ cmake \
    python3 tcl-devel swig bison flex git && \
    dnf clean all
```

3) macOS Binary Generation:

The existing **`github-actions-macos.yml`** only runs on forks macOS build failures go undetected on the main repository. I will remove this restriction and add binary packaging so macOS artifacts upload to GitHub Releases automatically on every version tag.

```yaml
jobs:
  Mac-Build:
    if: github.repository_owner != 'The-OpenROAD-Project' && github.repository_owner != 'The-O
    runs-on: macos-latest
    steps:
      - name: Setup xcode
        uses: maxim-lobanov/setup-xcode@v1
        with:
          xcode-version: latest-stable
      - name: Check out repository code
```

The `hw.ncpu` bug of `DependencyInstaller.sh` is already fixed in **PR #4030** , **macOS** now correctly detects logical cores using `hw.logicalcpu`.

For **`.deb`** and **`.rpm`** packages, matrix builds run inside Docker containers on **`ubuntu-latest`** runners. For **`.dmg`** (macOS), GitHub-hosted **macos-latest** runners are used directly since **macOS Docker images** do not exist. This bypasses the macOS VM limitation entirely while still producing all three package formats automatically on every version tag push.

4) Formal Versioning:

I will create a **VERSION** file in the repo root and `RELEASE.md` documenting the complete release process. Maintainers only need to push a tag so, everything else runs automatically

```
# Release Process

## Versioning: MAJOR.MINOR.PATCH
- MAJOR: breaking changes
- MINOR: new features, new platform
- PATCH: bug fixes

## Steps
1. Update VERSION file
2. Push tag: git tag v1.0.0
3. Release workflow triggers automatically
4. Binaries uploaded to GitHub Releases
5. Docker images published to GHCR
```

5) Automated Smoke Testing:

Before I allow any **artifact to upload to GitHub Releases**, I will trigger a mandatory **smoke test on a fresh clean runner**. I will run **`openroad -version`** to verify dynamic library linking and execute the **gcd design** to verify **functional routing**. If either step fails, I will hard-abort the workflow **,a broken binary will never reach the public.**

```yaml
smoke-test:
  needs: build
  runs-on: ubuntu-latest
  steps:
    - name: Download artifact
      uses: actions/download-artifact@v3

    - name: Install binary
      run: sudo dpkg -i openroad_ubuntu22.deb

    - name: Verify binary linking
      run: openroad -version

    - name: Run GCD smoke test
      run: |
        cd flow
        make DESIGN_CONFIG=./designs/nangate45/gcd/config.mk

    - name: Hard abort on failure
      if: failure()
      run: |
        echo "Smoke test failed. Blocking release upload."
        exit 1
```

6) CI Optimization:

Caching & Runner Limits **`macos-latest` runners** are slow and hit timeout limits frequently. I will integrate **`ccache` and `actions/cache`** into all workflows so only delta changes recompile .not the full source tree. I will cache CMake build artifacts and dependency layers between runs to cut build times significantly.

```yaml
name: CI Cache

on: [push, pull_request]

jobs:
  build:
    runs-on: macos-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v3

      - name: Cache ccache
        uses: actions/cache@v3
        with:
          path: ~/.ccache
          key: ${{ runner.os }}-ccache-${{ github.sha }}
          restore-keys: |
            ${{ runner.os }}-ccache-

      - name: Cache CMake dependencies
        uses: actions/cache@v3
        with:
          path: build/_deps
          key: ${{ runner.os }}-deps-${{ hashFiles('CMakeLists.txt') }}
          restore-keys: |
            ${{ runner.os }}-deps-

      - name: Configure ccache
        run: |
          ccache --max-size=500M
          ccache --set-config=compiler_check=content
```

7) Security-First Workflow Permissions:

I will enforce strict least-privilege permissions across the entire release workflow explicitly requesting only `permissions: { contents: write }` for GitHub Releases and `permissions: { packages: write }` for **GHCR.** Nothing broader. This ensures a compromised dependency cannot silently touch source code.

```yaml
name: Security-First Release Workflow

on:
  push:
    tags:
      - 'v*.*.*'

permissions:
  contents: write
  packages: write

jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v3

      - name: Upload to GitHub Releases
        uses: softprops/action-gh-release@v1
        with:
          files: |
            build/*.deb
            build/*.rpm
            build/*.dmg

      - name: Push Docker image to GHCR
        run: |
          echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${{ github.actor }} --password-stdin
          docker push ghcr.io/${{ github.repository }}/openroad:latest
```

8) First-Time Contributor Automation Bot

I will build a GitHub Action that auto-comments **`QuickStart.md`**, build instructions, and the Discord link on every new issue or **PR from a first-time contributor**. I will configure it to detect **`good first issue`** labels and attach relevant **onboarding resources automatically.** Jack and Vitor will stop answering the same setup questions repeatedly. I will set it up once , it runs forever with **zero maintenance.**

```yaml
name: First Time Contributor Bot

on:
  issues:
    types: [opened]
  pull_request:
    types: [opened]

jobs:
  welcome:
    runs-on: ubuntu-latest
    steps:
      - name: Welcome first-time contributor
        uses: actions/github-script@v6
        with:
          script: |
            const author = context.payload.sender.login;
            const { data: issues } = await github.rest.issues.listForRepo({
              owner: context.repo.owner,
              repo: context.repo.repo,
              creator: author,
              state: 'all'
            });
            if (issues.length === 1) {
              await github.rest.issues.createComment({
                owner: context.repo.owner,
                repo: context.repo.repo,
                issue_number: context.issue.number,
```

9)Installation Documentation + Tutorial

I will create **`docs/user/QuickStart.md`** that is a single page showing the fastest path from installation to running the GCD example design. Three options: pre-built binary (10 minutes), Docker (15 minutes), build from source (2 hours). **Each path is tested and verified.**

```markdown
# QuickStart — Zero to First Design

## Option 1: Pre-built Binary (10 minutes)
wget https://github.com/.../openroad_ubuntu22.deb
sudo dpkg -i openroad_ubuntu22.deb

## Option 2: Docker (15 minutes)
docker pull ghcr.io/.../openroad/ubuntu22.04:latest
docker run -it ghcr.io/.../openroad/ubuntu22.04:latest

## Run first design
cd flow
make DESIGN_CONFIG=./designs/nangate45/gcd/config.mk
klayout results/nangate45/gcd/base/6_final.gds
```

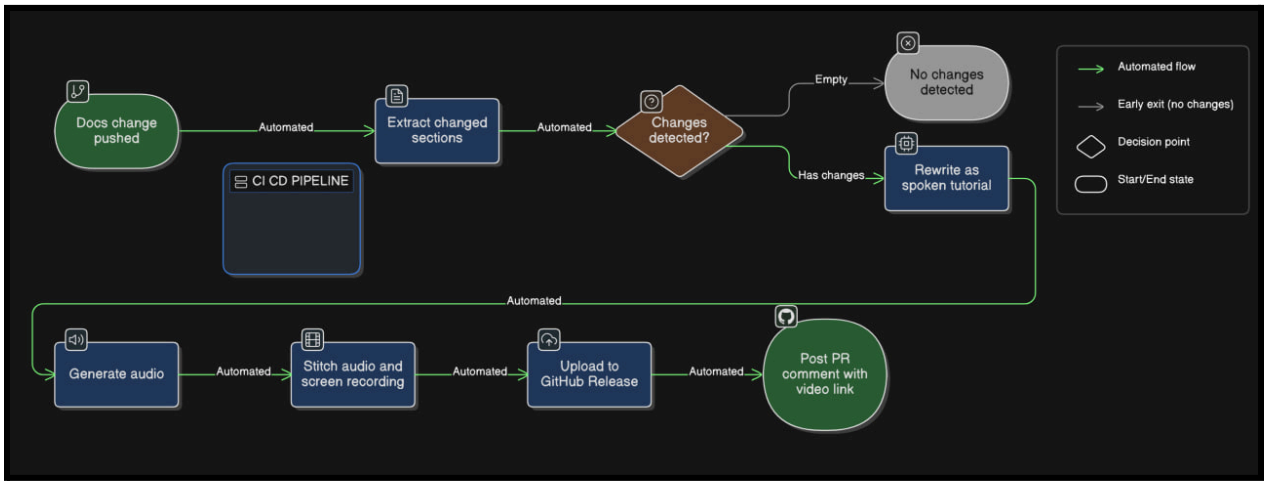
Stretch Goals

Stretch Goal 1: AI-Powered Tutorial Video Pipeline

OpenROAD's documentation changes frequently. Manually **re-recording tutorial videos** every time a doc is updated is unsustainable. **I will build a fully automated pipeline that regenerates tutorial videos on every documentation** change without any human intervention.

One problem nobody talks about with documentation is that **videos go stale**. A maintainer updates an installation step, but the tutorial video still shows the old flow. New users follow the video, hit a mismatch, and give up. This pipeline fixes that permanently.

The idea is simple that every time someone pushes a change to `docs/`, a GitHub Action wakes up, figures out exactly what changed, rewrites only that part of the narration, converts it to audio, and assembles a new video. No human touches it.



```yaml
name: Auto Tutorial Video Pipeline

on:
  push:
    paths:
      - 'docs/**'

jobs:
  regenerate-video:
    runs-on: self-hosted
    steps:
      - name: Checkout
        uses: actions/checkout@v3

      - name: Extract changed sections
        run: |
          git diff HEAD~1 HEAD -- docs/ > changed_sections.txt

      - name: Rewrite narration with llama.cpp
        run: |
          ./llama.cpp/main -m models/Q4_K_M.gguf \
          -p "You are a technical narrator. Rewrite this changed section as a clear spoken tutorial. No markdown, just speech." \
          -f changed_sections.txt > narration.txt

      - name: Generate audio with Coqui TTS
        run: |
          tts --text "$(cat narration.txt)" \
          --out_path audio.wav

      - name: Stitch video with FFmpeg
        run: |
          ffmpeg -i screen_recording.mp4 \
          -i audio.wav -c:v copy \
          -c:a aac tutorial_updated.mp4

      - name: Upload to GitHub Release
        uses: softprops/action-gh-release@v1
        with:
          files: tutorial_updated.mp4

      - name: Comment on PR
        uses: actions/github-script@v6
        with:
          script: |
            await github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.issue.number,
```

Here is how it actually works:

`git diff` isolates only the modified sections , there is no point reprocessing a 20-page doc because one paragraph changed. The pipeline runs on OpenROAD's existing self-hosted runners where RAM constraints are not a concern. For environments without self-hosted runners, I will use **`llama.cpp`** with a quantized 4-bit model (`Q4_K_M, ~2.5GB RAM`) which runs within GitHub's free runner limits without any API dependency or per-call cost. The prompt is straightforward: ***"You are a technical narrator. Rewrite this changed section as a clear spoken tutorial. No markdown, just speech."***

The generated script goes into **Coqui TTS**, which is fully open-source and self-hosted. I chose Coqui specifically because a pipeline that triggers dozens of times per week cannot depend on paid APIs. The audio comes out as **`.wav`**, FFmpeg stitches it with the existing screen recording, and the final **`.mp4`** gets uploaded directly to the GitHub Release.

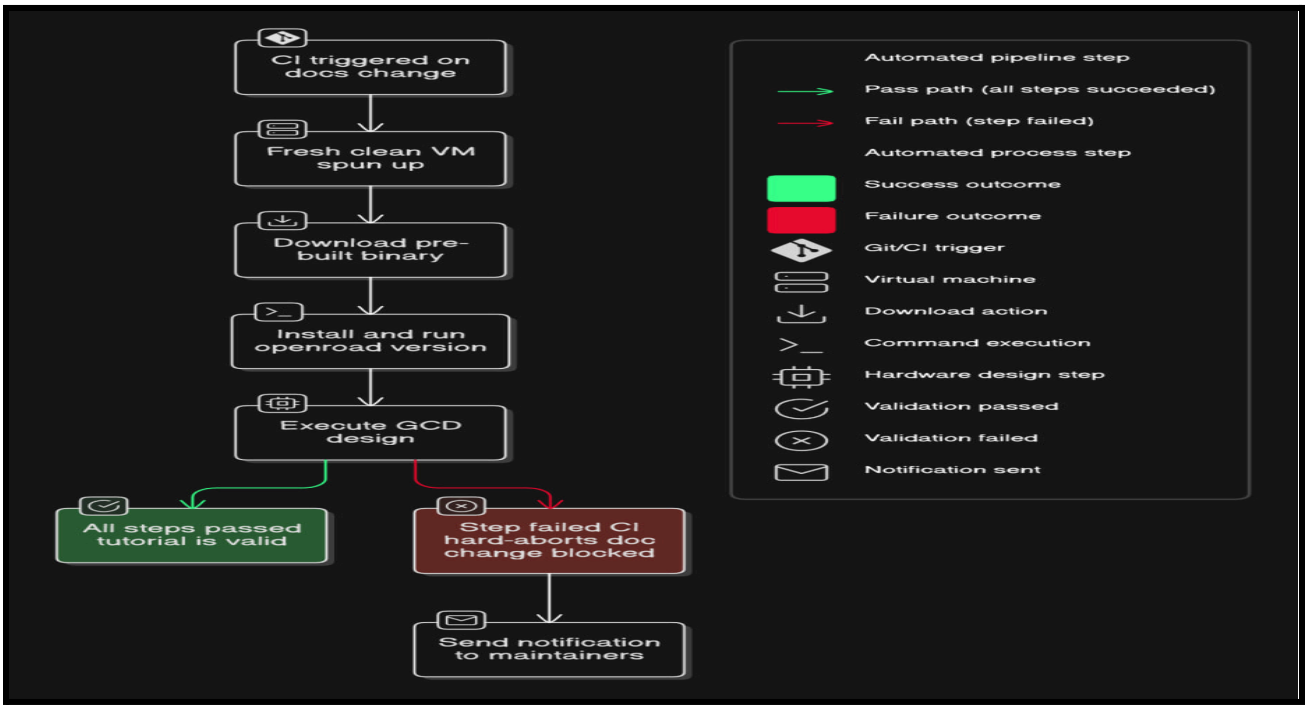
The maintainer who pushed the doc change gets a **GitHub comment on their PR** with a link to the new video. That is it. The video is always in sync with the docs, automatically, forever.

Why this matters:

Every time a **maintainer** updates installation docs, the tutorial video updates automatically. **No human re-recording. No stale videos. No paid APIs**. This would be the first automated doc-to-video pipeline in any open-source EDA project.

Stretch Goal 2: Tutorial Validation Automation Scripts

I will write `scripts/validate_tutorial.sh` that follows every step in `QuickStart.md` programmatically on a fresh environment and asserts expected outputs. Any future doc change that breaks the tutorial will be caught immediately in **CI ,no manual re-testing needed.**



Availability

My semester exams will be completed before the GSoC coding period starts in **June 2026.** I will be available **60 to 70** hours per week during the coding period. My timezone is **IST (UTC+5:30).** I am available daily from 6:00 PM to 6:00 AM IST on weekdays and flexible on weekends.

I have no planned vacations or commitments during the GSoC coding period. My preferred communication channels are Discord, Email, and GitHub comments. I will respond to mentor feedback **within 24 hours.**

Timeline

Community Bonding Period:

May 1 - May 25 :

I will reach out to **Jack and Vitor, set up weekly calls**, and **spend this time going through every installation path myself on a fresh VM**. I want to know every f**ailure point before I write a single line of code.**

Official Coding Period:

Week 1: May 26 - June 1:

I'll start by creating the **release workflow file**. The existing `buildDependenciesImage` job is the foundation . I just need to add the version tag trigger on top of it.

Week 2: June 2 - June 8:

Rocky Linux support. I'll create `docker/Dockerfile.rockylinux` and add it to the OS matrix. The pattern is identical to the existing **Ubuntu Dockerfile** just dnf instead of apt.

Week 3: June 9 - June 15:

Binary packaging. I'll make **`.tar.gz`** and .deb packages for Ubuntu and Rocky Linux and test uploading them to GitHub Releases.

Week 4: June 16 - June 22:

Automated changelog generation and build cadence setup. I will implement a **`generate_changelog.sh`** script that runs **`git log --pretty=format`** between two version tags, filters merge commits, and groups entries by type (fix, feat, build) following conventional commit prefixes. Output is written directly to **`CHANGELOG.md`** and attached to the GitHub Release body automatically.

Build cadences will be configured as three separate workflow triggers:

- **Nightly** runs on **`cron: '0 2 * * *'`**, builds from `main`, tags the artifact as **`nightly-YYYYMMDD`**, uploads to a rolling GitHub Release named **`nightly-latest`** (overwrites previous)
- **Stable** triggers on version tags matching `v*.*.*`, full matrix build across Ubuntu 20.04, Ubuntu 22.04, Rocky Linux 8/9, macOS, uploads versioned binaries to a permanent GitHub Release
- **Release** same as stable but additionally publishes Docker images to GHCR with both **`vX.Y.Z`** and **`latest`** tags

This separates development snapshots from production releases cleanly, so users can always find the latest stable binary without wading through nightly artifacts.

Week 5: June 23 - June 29:

Full end-to-end test. Push a real version tag and verify the binary shows up on GitHub Releases automatically.

Week 6: June 30 - July 6:

Buffer week. Fix anything that broke during testing. Write documentation for maintainers explaining how the pipeline works.

Midterm Evaluation: July 10

By this point the release pipeline will be fully working. **Rocky Linux** binaries will be on GitHub Releases. Changelogs will be generated automatically on every release.

Week 7: July 11 - July 17:

macOS binary generation. Remove the fork-only restriction from `github-actions-macos.yml` so macOS builds run on the main repo too.

Week 8: July 18 - July 24:

Expand the Windows Docker guide from **PR #4031.** Create the VERSION file and `RELEASE.md`.

Week 9: July 25 - July 31:

Publish versioned Docker images to **GHCR.** Add version compatibility matrix to installation docs.

Week 10: August 1 - August 7:

Update `BuildWithPrebuilt.md` for **macOS and Rocky Linux**. Fix `BuildWithDocker.md` with a proper troubleshooting section.

Week 11: August 8 - August 14:

Write `docs/user/QuickStart.md`. Test every installation path on a fresh VM if I can't follow my own tutorial, it's not good enough.

Week 12: August 15 - August 17:

Record the demo video. Address any remaining mentor feedback.

Final Evaluation: August 17 - August 24

The release pipeline will be running. Binaries for Ubuntu, Rocky Linux and macOS will be on GitHub Releases. VERSION file and `RELEASE.md` will be in the repo. `QuickStart.md` will be live with a demo video.

Extended Period: August 24 - November 2

If everything is done early I'll add **Debian 12 support**, build a version compatibility checker script and record platform-specific installation screencasts.

Final Extended Deadline: November 9

All work submitted and documented.

Milestones

Milestone 1 - Community Bonding Complete (May 25)

- All installation paths tested personally on fresh VMs (Ubuntu 20.04, 22.04, macOS, Rocky Linux 9)
- **Weekly sync cadence established with Jack and Vitor**
- Every known failure point documented before writing a single line of code

Milestone 2 - Midterm (July 10)

- **`github-actions-release.yml` live and triggering on version tags**
- Rocky Linux 8/9 binaries uploading automatically to GitHub Releases
- Automated changelog generating from `git log` on every release
- **Nightly/stable/release build cadences running correctly**

Milestone 3 - Final Evaluation (August 17)

- **macOS `.dmg`** binaries on GitHub Releases automatically
- `VERSION` file and `RELEASE.md` merged into both repos
- `docs/user/QuickStart.md` live and validated on fresh VMs
- **Demo video recorded and linked in release**

Milestone 4 - Extended Period (November 9)

- **Debian 12 support** added to OS matrix
- `scripts/validate_tutorial.sh` running in CI
- Self-hosted documentation platform prototype evaluated and recommended

Deliverables

By the end of GSoC 2026:

1) **The first deliverable** is a fully automated release pipeline implemented as github-actions-release.yml in both OpenROAD and OpenROAD-flow-scripts repositories. This workflow triggers on version tags and produces pre-built binaries for **Ubuntu 20.04, Ubuntu 22.04, Rocky Linux 8, Rocky Linux 9, and macOS** uploaded automatically to GitHub Releases with generated changelogs.

2)**The second deliverable** is complete Rocky Linux support through a new `docker/Dockerfile.rockylinux` and `corresponding _installRockyLinux()` **function in `DependencyInstaller.sh` following the exact same pattern as the existing Ubuntu support .**

3)**The third deliverable** is production-ready macOS binary generation with the fork-only CI restriction removed from `github-actions-macos.yml` and the `hw.ncpu` **bug** of DependencyInstaller.sh fixed already addressed in **PR #4030**.

4) **The fourth deliverable** is i will add a version compatibility matrix to BuildWithPrebuilt.md showing which OpenROAD binary version works with which ORFS version , currently completely undocumented.

5) **The fifth deliverable** is a formal versioning system consisting of a VERSION file in the repository root and a `RELEASE.md` document that gives maintainers a clear repeatable process for cutting releases.

6) **The sixth deliverable** is updated installation documentation across BuildWithPrebuilt.md, BuildWithDocker.md and **`BuildWithWSL.md`** covering all supported platforms with tested verified instructions and a version compatibility matrix

7) **The seventh deliverable** is a `docs/user/QuickStart.md` tutorial guiding a brand new user from zero to running their first complete **GDSII** layout using the GCD example design covering p**re-built binary, Docker, and build-from-source** paths with a demo video.

8) **The eighth deliverable is a First-Time Contributor Automation Bot, a GitHub** Action that automatically comments `QuickStart.md` link, build instructions, and Discord channel on every

new issue and PR opened by a first-time contributor, reducing repetitive mentor overhead to zero.

Metrics for Success

- **Time to First Design:** 2+ hours → under 10 minutes via pre-built binaries or Docker
- **CI Coverage:** 100% automated across Ubuntu, Rocky Linux, macOS, and WSL2/Windows
- **Release Reliability:** Zero broken releases — every binary passes clean-VM smoke test before publishing
- **Onboarding:** New user routes `gcd` via `QuickStart.md` with zero undocumented steps or missing dependencies

Tutorial Validation on Fresh VMs

Every installation path in `QuickStart.md` ,pre-built binary, Docker, and build-from-source will be validated on fresh VMs: **Ubuntu 22.04, Rocky Linux 9, and macOS** before final submission. I will follow the tutorial exactly as a new user would, with no prior setup. Terminal output will be screen-recorded as proof of completion. **If I cannot follow my own tutorial on a clean machine, it is not ready.**

Why Me?

I chose this project because I lived every single pain point in this proposal. When I first tried to set up **OpenROAD** I hit **every problem documented here like missing dependencies, broken macOS scripts, no Windows guide, no beginner tutorial.** I didn't complain about it. **I fixed it.**
Every single contribution in my prior contributions list came from a real problem I personally encountered and fixed. I experienced them and solved them.
I also have **5 merged PRs across both OpenROAD and OpenROAD-flow-scripts** repositories covering build **infrastructure, macOS fixes, Docker, documentation, and CI/CD.** Jack and Vitor have already reviewed my code. They know my work quality.

Future Scope

The 8 deliverables in this proposal are the floor, not the ceiling. If I finish ahead of schedule and given that I already know every failure point in this codebase from months of contributing, I expect here is exactly what comes next.

Debian 12 Support :

Rocky Linux was the hardest platform to add because it required a new Dockerfile pattern and a new function in `DependencyInstaller.sh`. **Debian 12 is easier** ,it shares the same apt package manager as Ubuntu, so the Dockerfile diff will be minimal.

The real work is validation: making sure every dependency version that works **on Ubuntu 22.04 also resolves correctly on Debian 12.** I will add it to the OS matrix and verify it end-to-end.

Self-Hosted Documentation Platform :

Right now OpenROAD's docs live across scattered markdown files with no unified search, no versioning, and no structured navigation. Vitor raised this in the office hours meeting , there is no self-hosted documentation platform.

I will evaluate three candidates:

- **MkDocs Material**
- **Antora**
- **Docusaurus.**

My evaluation criteria will be, does it support versioned docs, can it be self-hosted with zero external dependencies, and can it be deployed automatically on every release tag.

I will prototype all three during the extended period and recommend the best fit based on actual testing, not just feature comparisons.

Expanding the AI Video Pipeline:

The stretch goal covers installation docs. The natural next step is expanding it to the full ORFS FlowTutorial , the **10-stage RTL-to-GDSII flow.** Each stage already has a corresponding markdown file.

The pipeline architecture is identical , git diff triggers regeneration, llama.cpp rewrites the narration, Coqui TTS generates audio, FFmpeg assembles the video.

The only additional work is capturing screen recordings for each flow stage, which I will do on a fresh machine running the nangate45 GCD example.

Version Compatibility Checker Script:

Right now there is no way for a user to know if their installed OpenROAD binary version is compatible with their ORFS version.

I will write **`scripts/check_compatibility.sh`** that is a script that reads the local OpenROAD version, queries the compatibility matrix, and tells the user immediately if their setup is supported or not.

One command, instant answer.

Docker Image Namespace Migration:

Currently, Docker images are published under a personal namespace. Moving them to the **`The-OpenROAD-Project`** organizational namespace on GHCR is a planned future task confirmed during mentor office hours. This makes images officially discoverable under the project's canonical namespace and removes any dependency on individual contributor accounts.

The change itself is straightforward , a single namespace update in the existing **`github-actions-publish-docker-images.yml`** workflow.

About Me

I am a **final-year Computer Science undergraduate** at **JIS University**, Kolkata,India. My **open source journey** started in my very **first semester** when a senior who had contributed to **GSoC** showed me what it meant to build something that actually mattered to people outside your own laptop. **That conversation rewired how I thought about software**.

I did not ease into open source. I started with **Summer of Bitcoin**, contributed to **GirlScript Summer of Code twice**, and then switched sides , **becoming a mentor**. In my 4th semester I was **selected** as an open source mentor for JGEC students, guiding contributors on **DivineShield, a women's safety application** I founded and continue to build.

Teaching others how to contribute taught me more about software quality than any course ever did.

I also built AutoFlow, an **RTL2GDS orchestration engine with a C++ DAG scheduler and TCL** scripting layer mirroring OpenROAD's pipeline architecture , giving me deep familiarity with the tool chain before I ever opened the codebase.

I am a two-time ICPC Regionalist. **Competitive programming** trained me to read codebases fast, reason about edge cases systematically, and **fix bugs** under pressure ,exactly the skills that showed up in every one of my OpenROAD contributions.

Why this project as a CSE student?

The honest answer is because I hit every single problem in this proposal myself. I am a **CSE** student, **not an EDA or VLSI specialist**. But when I first tried to set up OpenROAD for a project, I spent hours **fighting missing dependencies, a broken macOS installer, and documentation that assumed you already knew what you were doing**. I did not have a chip design background. I just had a broken build and no guide to fix it.

That frustration is exactly why this project matters. OpenROAD is used by researchers and students worldwide , most of whom, like me, are not EDA specialists. They are CSE students, hobbyists, and academics who just want to run a design. Right now, most of them give up before running a single one.

A CSE background is not a limitation here , it is the exact perspective this project needs. I know what it feels like to be a new user hitting these problems because I was that user. And instead of giving up, I fixed it.

On the technical side, I am experienced in C++, CMake, GitHub Actions, Docker, and shell scripting. I have diagnosed and fixed real build failures across macOS, Ubuntu, and Windows environments.

I do not read about infrastructure problems and theorize solutions. **I reproduce them**, trace them to their root cause, and submit a fix.That is how **PR #9805 happened.** That is how **PR #4030** happened. That is how all 5 of my merged PRs happened.

I am not applying to **OpenROAD because** it looks good on a resume. I am applying because I have been **contributing to it for months**, I know exactly where it is broken, and I have a clear plan to fix it.